

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1- Constraint-Based Problem Solving
Weightage:	40%	Total Marks:	25
CO5 Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Kaviyaasri M	Reg.No.:	192312614
Department:	ECE	Date:	

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s

- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Real-Time Embedded System I/O

Communication

- Constraints
- CPU utilization < 15%
- Multiple sensors interrupt every 1 ms
- Vectored interrupt handling
- Choose memory-mapped or I/O-mapped I/O for low latency

Solution

Use memory-mapped I/O (MMIO) with a vectored interrupt controller and efficient interrupt service routines.

Architecture:

Sensors → Interrupt Controller → CPU

Sensors → MMIO Registers → CPU/Memory

Working

- Each sensor is connected to an interrupt line.
- The interrupt controller identifies the sensor using vectored interrupts.
- CPU directly accesses sensor registers using memory-mapped I/O.
- ISR performs only essential processing and stores data in a buffer.
- Longer processing is done later as a background task.
- This reduces CPU time and keeps utilization below 15%.

Why MMIO?

MMIO provides simple and fast access to device registers using normal CPU instructions, giving low I/O latency.

Comparison

Memory-Mapped I/O	I/O-Mapped I/O
Uses normal memory instructions	Uses special I/O instructions
Large address space	Separate I/O address space
Simple programming	Requires special instructions
Usually offers convenient/low-latency access	Can add instruction/addressing restrictions
Selected for this system	Not selected

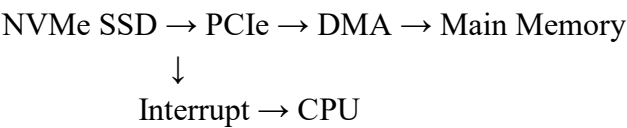
Result: Fast sensor response with low CPU utilization.

2. High-Speed Storage I/O Subsystem

Constraints

- Throughput > 3 GB/s
- NVMe over PCIe
- CPU should not handle bulk data
- DMA with interrupt notification
- Solution
- Use an NVMe SSD connected through PCIe, with DMA for transferring data directly between SSD and main memory.

Architecture:



Working

- CPU sends a read/write command to the NVMe controller.
- NVMe communicates through the PCIe bus.
- DMA transfers bulk data directly between the SSD and RAM.
- CPU is not involved in every byte/word transfer.
- After completion, the NVMe controller generates an interrupt.
- CPU handles the completion notification.

Advantages

- High throughput (>3 GB/s)
- Very low CPU overhead
- Efficient bulk data transfer
- PCIe provides high-speed communication.
- Result: A high-performance storage subsystem with efficient CPU usage.

Comparison

Technique	Role in the System	Advantage
NVMe	Connects to high-speed SSD	Very high storage performance
PCIe	High-speed interface for NVMe	High bandwidth, low latency
DMA	Transfers data directly between SSD and RAM	Reduces CPU involvement
Interrupt	Notifies CPU after transfer	Efficient CPU notification
CPU	Controls transfer and processes completion	Avoids bulk data handling

Result

The proposed NVMe over PCIe subsystem with DMA provides high-speed data transfer above 3 GB/s while minimizing CPU involvement. The DMA controller handles bulk data transfer, and interrupts notify the CPU after completion, resulting in efficient and high-performance I/O communication.

3. Multi-Device Communication Architecture

Constraints

- 6 USB peripherals
- 2 high-speed Thunderbolt devices
- Avoid interrupt starvation
- Hardware + software interrupts
- Fair bus arbitration

Solution

Use a USB hub/controller for the six USB devices and a Thunderbolt controller for the two high-speed devices.

Architecture:

6 USB Devices → USB Controller → Interrupt Controller → CPU 2 Thunderbolt Devices → Thunderbolt Controller → Interrupt Controller → CPU

Working

- USB devices share the USB controller.
- Thunderbolt devices use the high-speed Thunderbolt controller.
- Hardware interrupts are used for urgent device events.
- Software interrupts handle lower-priority/background processing.
- A priority + round-robin arbitration scheme prevents one device from continuously occupying the bus.
- Interrupt priorities and fair scheduling prevent interrupt starvation.

Comparison

Component	Function	Benefit
USB Controller	Connects 6 USB peripherals	Supports multiple devices
Thunderbolt Controller	Connects 2 high-speed devices	High-speed communication
Hardware Interrupt	Handles urgent device events	Fast response
Software Interrupt	Handles background tasks	Reduces hardware interrupt load
Bus Arbitration	Shares bus among devices	Prevents starvation and ensures fairness

Result:

All eight devices get fair access to system resources while high-priority devices receive timely service.

4. Data Acquisition System**Constraints**

- Continuous data stream = 500 MB/s
- Minimal CPU intervention
- Compare DMA and interrupt-driven I/O
- Memory-mapped I/O for device registers
- Solution
- Use DMA for continuous data transfer and MMIO for controlling the data acquisition device.

Architecture:

Sensor → Data Acquisition Device → DMA → RAM
↓
Interrupt → CPU

DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Large data transfer	Excellent	Less suitable
500 MB/s stream	Suitable	Not efficient
CPU overhead	Low	High
Performance	High	Lower

Working

- CPU configures the device registers using MMIO.
- DMA controller is configured with the destination memory address.
- The device continuously generates data.
- DMA transfers data directly to RAM.
- CPU receives an interrupt after a block of data is transferred.
- CPU processes the completed block while DMA continues receiving new data.

Result:

DMA is preferred because it supports 500 MB/s continuous transfer with minimal CPU intervention.

5. Nested Interrupt Handling Mechanism

Constraints

- Support nested interrupts
- Maximum latency < 10 μ s
- Vectored interrupts for high-priority devices
- Software interrupts for background tasks

Solution

Use a priority-based interrupt controller with nested interrupt support.

Architecture:

High-Priority Device $\rightarrow$ Vectored Interrupt $\rightarrow$ CPU

Low-Priority Device $\rightarrow$ Interrupt Controller $\rightarrow$ CPU

Background Tasks $\rightarrow$ Software Interrupt $\rightarrow$ CPU

Working

- Each interrupt is assigned a priority.
- High-priority devices use vectored interrupts for fast identification.
- When a high-priority interrupt arrives, it can interrupt a lower-priority ISR.
- The CPU saves the current ISR state and services the higher-priority interrupt.
- After completion, the CPU returns to the interrupted ISR.
- Background tasks use software interrupts, so they do not delay critical hardware events.
- Short ISRs and fast vector lookup help keep interrupt latency below 10 μ s.

Simple Priority

Highest $\rightarrow$ Critical hardware interrupt

$\downarrow$

Normal interrupt

$\downarrow$

Lowest $\rightarrow$ Software/background interrupt

Interrupt Type	Usage	Priority/Benefit
Vectored Interrupt	High-priority devices	Fast ISR identification
Hardware Interrupt	External device events	Fast response
Software Interrupt	Background tasks	Controlled software processing
Nested Interrupt	Higher-priority interrupt during ISR	Reduces critical-event latency
Priority Controller	Manages interrupt priority	Helps maintain < 10 μ s latency

Result:

Nested, priority-based interrupt handling provides fast response for critical devices while allowing background tasks to run efficiently.